

FEATURE ENGINEERING — REVISION CHEAT SHEET

Process of preparing & transforming raw data into informative features that improve ML model performance.

Feature Engineering = Transformation + Construction + Selection + Extraction

- Transformation: impute / encode / scale / handle outliers
- Construction: create new features from existing ones
- Selection: keep only most relevant features
- Extraction: reduce dimensions e.g. PCA

1. HANDLING MISSING VALUES

Handle before: scaling, encoding, feature selection, model training.

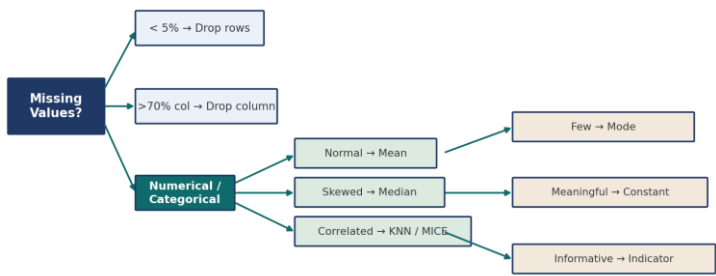
Types of Missingness

| Type | Meaning                    | Fix                                  |
|------|----------------------------|--------------------------------------|
| MCAR | Completely random          | Drop rows / Mean-Median-Mode         |
| MAR  | Depends on another feature | KNN, MICE, related-feature impute    |
| MNAR | Depends on value itself    | Domain knowledge / Missing indicator |

Method — When to use

| Method               | Use when                       |
|----------------------|--------------------------------|
| Drop rows            | < 5% missing                   |
| Drop column          | > 60–70% missing & unimportant |
| Mean                 | Numerical + normal dist.       |
| Median               | Numerical + skewed / outliers  |
| Mode                 | Categorical                    |
| Constant ("Unknown") | Missing itself has meaning     |
| Fwd / Bwd fill       | Time series                    |
| KNN Imputer          | Correlated features            |
| Iterative (MICE)     | Multiple correlated features   |
| Missing Indicator    | Missingness pattern is useful  |

Decision Flow



Algorithm Compatibility

| Handles missing directly    | Needs imputation first                                                   |
|-----------------------------|--------------------------------------------------------------------------|
| XGBoost, LightGBM, CatBoost | Linear/Logistic Reg, SVM, KNN, Neural Nets, Decision Tree, Random Forest |

2. MULTIVARIATE IMPUTATION

Missing values of one feature estimated using info from multiple other features.

KNN Imputer

Finds K most similar rows (Nan-Euclidean distance) & averages their known values to fill the gap.

$$d(x,y)=\sqrt{\frac{m}{m_{\text{obs}}}\sum_{i\in\text{observed}}(x_i-y_i)^2}$$

- Pros: uses feature relationships, more accurate
- Cons: costly, needs scaling, sensitive to K

Data w/ NaN -> Find K-nearest rows -> Avg known values -> Replace NaN

MICE (Chained Equations)

Iterative, model-based: predicts each missing feature using the others in a chain, repeating until values stabilise.

Mean-impute all -> Model predicts col A -> then col B... -> repeat till convergence

| Missing type | MICE suited? |
|--------------|--------------|
| MCAR         | Yes          |
| MAR          | Best suited  |
| MNAR         | Generally no |

- Pros: uses relationships, flexible, improves w/ iterations
- Cons: expensive, model-sensitive, may not converge

3. HANDLING MIXED VARIABLES

A single column holding both numeric & categorical info, e.g. Cabin 'C22' -> Deck 'C' + No. 22.

- Fix: split into two separate columns before encoding/scaling.
- Left as-is -> hides patterns, blocks scaling/encoding.

4. WORKING WITH DATES & TIME

Raw date -> Convert to datetime -> Extract useful parts

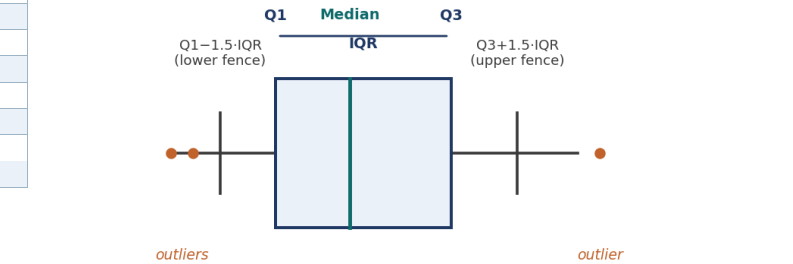
| Date parts                | Time parts                  | Derived                                      |
|---------------------------|-----------------------------|----------------------------------------------|
| Year, Month, Day, Weekday | Hour, Minute, Second, AM/PM | Date-diff, is_holiday, is_weekend, month-end |

HANDLING OUTLIERS

5-number summary -> shows spread of data, detects outliers, builds box-plot.

| Stat        | Meaning                  |
|-------------|--------------------------|
| Min / Max   | Smallest / Largest value |
| Q1 (25%)    | 25% of data below        |
| Median (Q2) | Middle value (50%)       |
| Q3 (75%)    | 75% of data below        |

IQR = Q3-Q1    Lower fence = Q1-1.5·IQR    Upper fence = Q3+1.5·IQR



IQR is preferred over range — resistant to outliers.

Detection / Treatment Methods

| Method             | Best for               |
|--------------------|------------------------|
| IQR method         | Skewed data            |
| Z-score            | Normal distribution    |
| Box plot           | Visualisation          |
| Percentile capping | Extreme values         |
| Winsorization      | Replace extreme values |

- Ignoring outliers distorts the mean, inflates variance & reduces model accuracy.
- Tree-based models are largely robust to outliers/skew.

6. HANDLING IMBALANCED DATASET

| Imbalance | Ratio ex. | Best approach                     |
|-----------|-----------|-----------------------------------|
| Mild      | 600:400   | Usually no special handling       |
| Moderate  | 900:100   | Class weights / Resampling        |
| Severe    | 9900:100  | SMOTE / Ensemble / Cost-sensitive |

| Method                | Use when                          |
|-----------------------|-----------------------------------|
| Random under-sampling | Large majority class              |
| Random over-sampling  | Small minority class              |
| Class weights         | Algo supports weights             |
| Threshold tuning      | Need higher recall                |
| SMOTE                 | Create synthetic minority samples |
| Ensemble methods      | Severe imbalance                  |

- Never resample before train-test split -> causes data leakage.
- Not used for regression problems.

SMOTE — Synthetic Minority Over-sampling

Pick sample -> Find K-nearest nbrs -> Choose neighbour -> Interpolate

$$x_{new} = x_i + \lambda(x_{NN} - x_i)$$

$$\lambda \in (0,1)$$



- Creates new synthetic points (not duplicates) → fixes over-sampling's lack of variance.

## 7. ENCODING CATEGORICAL DATA

| Encoding              | Best for                      |
|-----------------------|-------------------------------|
| Label                 | Target labels                 |
| Ordinal               | Ordered categories            |
| One-Hot (OHE)         | Nominal categories            |
| Target-guided ordinal | High-card., correl. w/ target |
| Frequency / Count     | Category frequency matters    |
| Binary / Hash         | High-cardinality / big data   |

`OneHot(xi)=1 if xi=Cj else 0` | Ordinal: order = mean-target rank

- OHE → vectors orthogonal, no false ordinal relationship created.
- Target-guided: rank categories by mean(target), encode by rank.

## 8. FEATURE SCALING

|               | Standardization          | Normalization                |
|---------------|--------------------------|------------------------------|
| Formula       | $z = (x - \mu) / \sigma$ | $(x - \min) / (\max - \min)$ |
| Output        | mean 0, std 1            | range [0,1]                  |
| Outlier sens. | Medium                   | High (Robust=low)            |

- Normalization types: Min-Max, Mean-norm, MaxAbs, Robust (uses IQR — least outlier-sensitive).
- Scaling never removes outliers, only rescales.

*Outliers? Yes → Robust → No → Normal? Yes → Standardize → No → check algo needs*

## 9. DATA TRANSFORMATION (SKEW → NORMAL)

Symmetric

Mean=Median=Mode



Right-skewed (+ve)

Mean>Median>Mode



Left-skewed (-ve)

Mean<Median<Mode



| Skew        | Mean vs Median   | Fix                        |
|-------------|------------------|----------------------------|
| Symmetric   | Mean=Median=Mode | —                          |
| +ve (right) | Mean>Median>Mode | Log, Sqrt, Recip., Box-Cox |
| -ve (left)  | Mean<Median<Mode | Square, Cube, Yeo-Johnson  |

Box-Cox:  $x' = (x^{\lambda} - 1) / \lambda$ ,  $x > 0$  only | Yeo-Johnson: handles 0 & negatives too

$$y(\lambda) = \begin{cases} \frac{x^{\lambda} - 1}{\lambda}, & \lambda \neq 0 \\ \ln(x), & \lambda = 0 \end{cases}$$

- Power Transformer auto-picks best  $\lambda$  → most normal-like, reduces skew & outlier effect.

## 10. BINNING & BINARIZATION

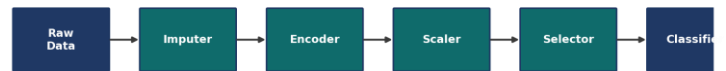
| Type            | Rule                                          |
|-----------------|-----------------------------------------------|
| Equal width     | bin width = $(\max - \min) / n_{\text{bins}}$ |
| Equal frequency | each bin ~same % of data                      |
| K-means binning | cluster-based bins                            |
| Binarization    | $x' = 1$ if $x > \text{threshold}$ else 0     |

- Unsupervised binning is sensitive to outliers; custom binning uses domain knowledge.

## 11. COLUMN TRANSFORMER & ML PIPELINE

`X' = [ T1(X_num), T2(X_cat), ..., Tk(X_subset) ]` (outputs concatenated)

Applies different preprocessing to different column subsets in one step — avoids repeated code & leakage.

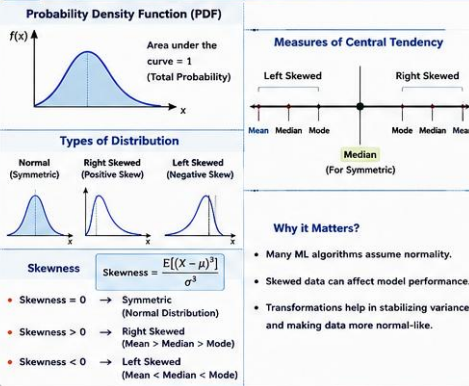


- Never fit() on test data — only transform() — to avoid data leakage.
- Pipeline = chain of transformers + final estimator; each step feeds the next.

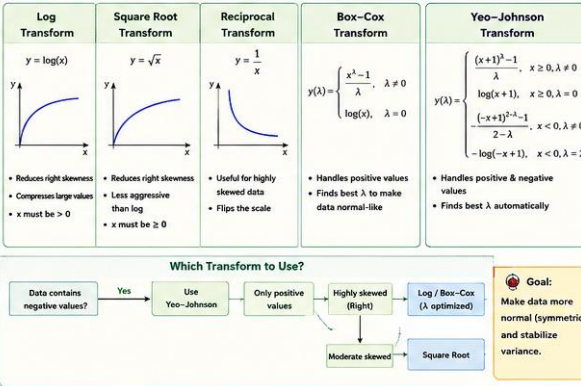
# FEATURE ENGINEERING - PART 1 (MASTER CHEATSHEET)

TRANSFORMS • MISSING DATA → DISTRIBUTIONS • DISCRETIZATION → MIXED VARIABLES • DATE & TIME

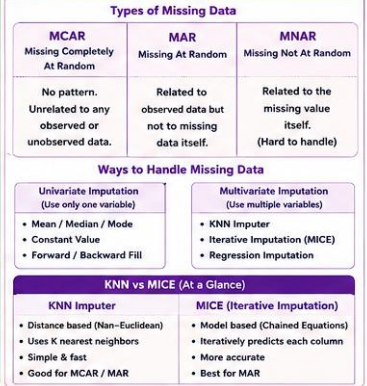
## 1 PROBABILITY DISTRIBUTION & SKEWNESS



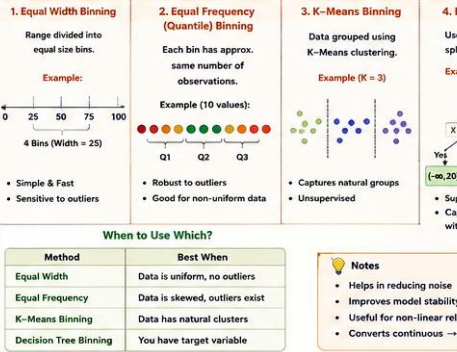
## 2 FUNCTION TRANSFORMERS (WHY & HOW)



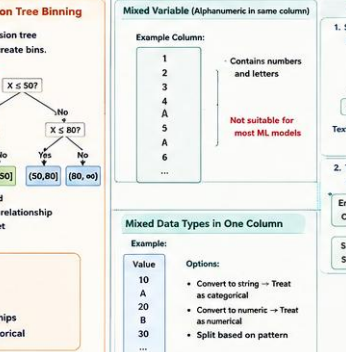
## 3 MISSING DATA



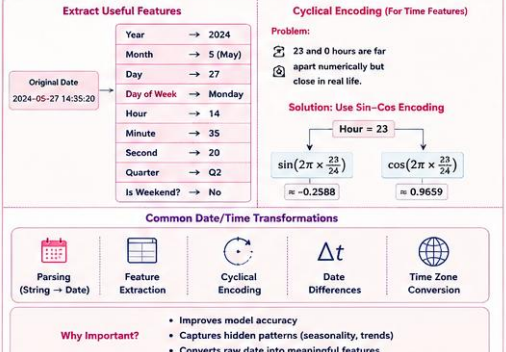
## 4 DISCRETIZATION / BINNING (Turning Continuous → Categorical)



## 5 HANDLING MIXED VARIABLES



## 6 HANDLING DATE & TIME VARIABLES



**Key Takeaways**

- Understand the nature of your data (Distribution, Missingness, Type)
- Transform smartly (not always log!)
- Create meaningful categories (Binning)
- Extract time intelligence (Dates & Times)
- Always validate after feature engineering!

**Remember**

- Better Features → Better Models
- Domain Knowledge + EDA = Powerful Features
- Visualize, Transform, Encode, Validate

**Legend**

- Concept: Concept, Method: Method, Tip/Note: Tip/Note, Workflow: Workflow

**Symbols Used**

- Start / End: Start / End, Decision: Decision, Process: Process, Flow: Flow

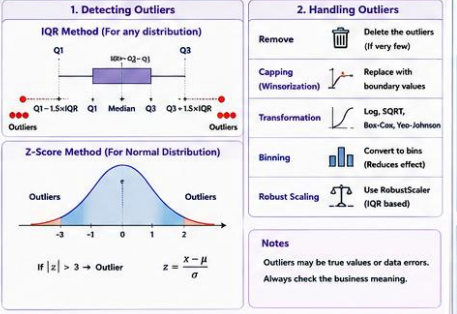
**QR Code**

Scan to Revise All Concepts

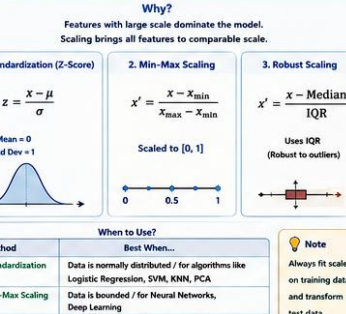
# FEATURE ENGINEERING - PART 2 (MASTER CHEATSHEET)

OUTLIER HANDLING • FEATURE SCALING • ENCODING TECHNIQUES • FEATURE TRANSFORMATION • FEATURE SELECTION • INTERACTION FEATURES

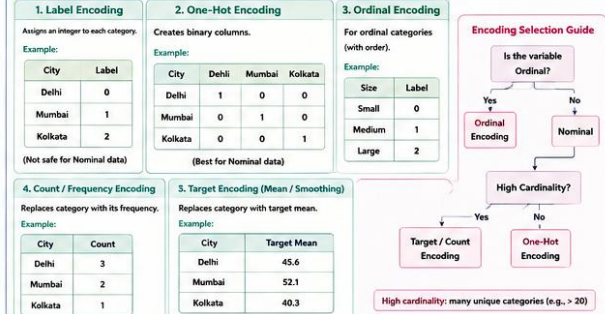
## 7 OUTLIER HANDLING



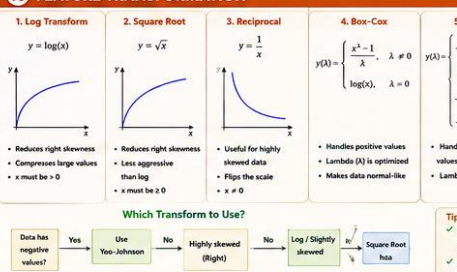
## 8 FEATURE SCALING



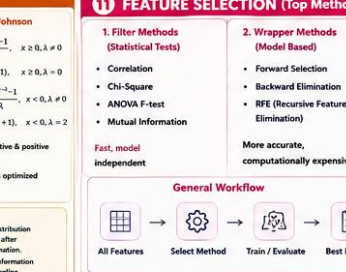
## 9 ENCODING TECHNIQUES



## 10 FEATURE TRANSFORMATION



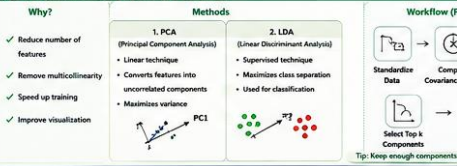
## 11 FEATURE SELECTION (Top Methods)



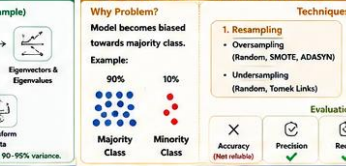
## 12 INTERACTION FEATURES



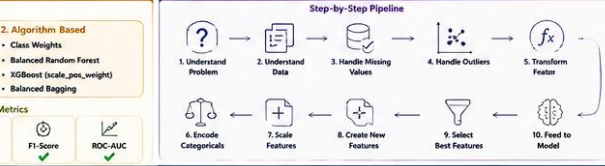
## 13 DIMENSIONALITY REDUCTION



## 14 HANDLING CLASS IMBALANCE



## 15 FEATURE ENGINEERING PIPELINE



**KEY TAKEAWAYS**

- Understand your data deeply.
- Apply the right transformation.
- Use proper encoding.
- Handle missing values smartly.
- Select important features.
- Always validate your results.

**REMEMBER**

- Better Features → Better Models
- Good Feature Engineering = 50% of a Good ML Model

**LEGEND**

- Concept: Concept, Method: Method, Tip/Note: Tip/Note, Example: Example, Caution: Caution

**SCAN TO REVISE**

Keep learning and keep building!